FIAP — Análise e Desenvolvimento de Sistemas
Fase 6 — DevOps

EcoTracker

Projeto Cidades ESG Inteligentes

Documentação técnica da entrega de DevOps:
CI/CD, containerização, orquestração e evidências de deploy

Grupo 102

Markus de Figueiredo Ferraz Wirz — markusfwirz@gmail.com

Outubro de 2026

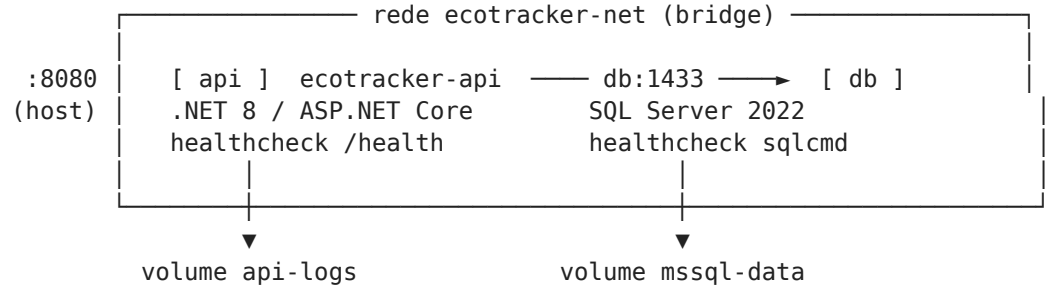
1. O projeto

O EcoTracker é uma API RESTful em .NET 8 voltada à governança e ao compliance ambiental (ESG) de cidades e organizações. Ela monitora emissões de carbono e compensações, controla licenças ambientais e seus prazos de renovação, registra auditorias de conformidade e dispara alertas ambientais. A aplicação já existia nas fases anteriores do curso; esta entrega acrescenta a camada de DevOps: containerização, orquestração e um pipeline de CI/CD com deploy automatizado em staging e produção.

Camada	Tecnologia
Aplicação	.NET 8 / ASP.NET Core, EF Core 8, autenticação JWT, Swagger/OpenAPI
Testes	xUnit — 17 testes de integração com WebApplicationFactory
Container	Docker, build multi-stage, imagem final mcr.microsoft.com/dotnet/aspnet:8.0
Orquestração	Docker Compose — serviços api + db, volumes, rede bridge e variáveis de ambiente
Banco de dados	SQL Server 2022 em container (fallback InMemory sem connection string)
CI/CD	GitHub Actions + composite action reutilizável de deploy
Registry	GitHub Container Registry (ghcr.io)

2. Arquitetura da solução em containers

A stack é composta por dois serviços em uma rede bridge dedicada. A API nunca acessa o banco pelo host: usa o hostname interno db da rede do Compose. O estado do banco e os logs da aplicação ficam em volumes nomeados, de modo que os containers possam ser recriados sem perda de dados.



Variáveis sensíveis (senha do SQL Server e chave JWT) vêm do arquivo .env — versionamos apenas o .env.example com valores de exemplo; no pipeline elas vêm dos secrets do repositório.

3. Dockerfile

O Dockerfile usa build multi-stage, separando compilação, testes, publicação e runtime. Isso mantém a imagem final pequena (243 MB) e sem o SDK .NET.

Estágio	Base	Função
build	sdk:8.0	restore em camada separada (cache de NuGet) + dotnet

		build -c Release
test	build	dotnet test dentro da imagem (estágio opcional)
publish	build	dotnet publish -c Release /p:UseAppHost=false
final	aspnet:8.0	apenas runtime + binários publicados, usuário não-root, EXPOSE 8080, HEALTHCHECK

```
docker build -t ecotracker-api:local .
docker run -p 8080:8080 -e DisableHttpsRedirection=true ecotracker-api:local
```

Para operar bem em container, a aplicação recebeu um endpoint anônimo GET /health (ASP.NET Core Health Checks) e três chaves de configuração: EnableSwagger (liga o Swagger fora de desenvolvimento), DisableHttpsRedirection (o container serve apenas HTTP, o TLS fica no proxy) e Database:StartupRetries (retentativas de conexão enquanto o SQL Server ainda está subindo).

4. Orquestração com Docker Compose

Recurso	Definição
Serviços	api (porta ${API_PORT} → 8080) e db (SQL Server 2022)
Volumes	mssql-data (dados do banco) e api-logs — por ambiente: mssql-data-staging / mssql-data-prod
Rede	ecotracker-net (bridge), isolando a comunicação api ↔ db
Variáveis	.env → ConnectionStrings__DefaultConnection, Jwt__Key, EnableSwagger, ASPNETCORE_ENVIRONMENT…
Saúde	healthcheck nos dois serviços; a api só inicia com o db service_healthy; restart unless-stopped

Os ambientes são arquivos de override aplicados sobre a mesma base:

```
# staging  — Swagger habilitado, porta 8080
docker compose -f docker-compose.yml -f docker-compose.staging.yml up -d

# produção — Swagger desabilitado, porta 9090, banco sem porta publicada, limites de
CPU/memória
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

5. Pipeline de CI/CD (GitHub Actions)

Arquivo .github/workflows/ci-cd.yml. Dispara em push em qualquer branch, em pull request e manualmente (workflow_dispatch). Os jobs são encadeados, de modo que uma falha de build ou de teste impede a publicação da imagem e os deploys.

```
build ⟶ test ⟶ docker ⟶ deploy-staging ⟶ deploy-production
```

Job	Etapas
build	dotnet restore, dotnet build -c Release e dotnet publish; sobe o artefato ecotracker-publish
test	dotnet test -c Release com relatório .trx publicado como artefato (17 testes)

docker	build multi-stage com cache do Buildx e push para ghcr.io nas tags :<sha> e :latest
deploy-staging	ambiente staging: sobe a stack com a imagem recém-publicada, aguarda /health e roda smoke test
deploy-production	ambiente production, só após o staging: mesmo fluxo com o override de produção na porta 9090

A lógica de deploy ficou isolada em uma composite action (.github/actions/deploy-compose) reutilizada pelos dois ambientes, variando apenas o override do Compose, a porta e os segredos. Cada deploy: faz login no registry, gera o .env do ambiente, sobe a stack, espera o /health responder e executa um smoke test que autentica via JWT e consulta um endpoint protegido — se qualquer etapa falhar, o job falha e os logs dos containers são impressos.

6. Evidências

6.1 Build e testes automatizados

```
$ dotnet build EcoTracker.sln -c Release
  Determining projects to restore...
  All projects are up-to-date for restore.
  EcoTracker -> /home/ubuntu/aspcore8/src/EcoTracker/bin/Release/net8.0/EcoTracker.dll
  EcoTracker.Tests -> /home/ubuntu/aspcore8/tests/EcoTracker.Tests/bin/Release/net8.0/EcoTracker.Tests.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:02.75
```

Figura 1 — Compilação da solução em Release, sem warnings.

```
$ dotnet test EcoTracker.sln -c Release
Test run for /home/ubuntu/aspcore8/tests/EcoTracker.Tests/bin/Release/net8.0/EcoTracker.Tests.dll (.NETCoreApp,Version=v8.0)
VSTest version 17.11.1 (x64)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed!  - Failed:     0, Passed:    17, Skipped:     0, Total:    17, Duration: 99 ms - EcoTracker.Tests.dll (net8.0)
```

Figura 2 — Suíte de testes: 17 aprovados, 0 falhas.

6.2 Imagem, volumes e rede

```
$ docker compose -f docker-compose.yml -f docker-compose.staging.yml up -d
 16e8436c0cba Extracting [==================================================> ]  91.91MB/93.03MB
 16e8436c0cba Extracting [==================================================>]  93.03MB/93.03MB
 16e8436c0cba Pull complete
 db Pulled
 Network ecotracker_ecotracker-net  Creating
 Network ecotracker_ecotracker-net  Created
 Volume "ecotracker_api-logs"  Creating
 Volume "ecotracker_api-logs"  Created
 Volume "ecotracker_mssql-data-staging"  Creating
 Volume "ecotracker_mssql-data-staging"  Created
 Container ecotracker-db-staging  Creating
 Container ecotracker-db-staging  Created
 Container ecotracker-api-staging  Creating
 Container ecotracker-api-staging  Created
 Container ecotracker-db-staging  Starting
 Container ecotracker-db-staging  Started
 Container ecotracker-db-staging  Waiting
 Container ecotracker-db-staging  Healthy
 Container ecotracker-api-staging  Starting
 Container ecotracker-api-staging  Started
```

Figura 3 — docker compose up: rede, volumes e containers criados; a API só sobe após o banco ficar saudável.

```
# Imagem, volumes e rede criados
$ docker compose -f docker-compose.yml -f docker-compose.staging.yml ps
NAME                   IMAGE                                      COMMAND                  SERVICE   CREATED          STATUS                    PO
ecotracker-api-staging   ecotracker-api:local                       "dotnet EcoTracker.d…"   api       22 seconds ago   Up 15 seconds (healthy)   0.
    ->8080/tcp, :::8080->8080/tcp
ecotracker-db-staging    mcr.microsoft.com/mssql/server:2022-latest   "/opt/mssql/bin/laun…"   db        22 seconds ago   Up 21 seconds (healthy)   0.
    ->1433/tcp, :::1433->1433/tcp

$ docker images ecotracker-api
REPOSITORY       TAG       IMAGE ID       CREATED          SIZE
ecotracker-api   local     8ad2211524d8   48 seconds ago   243MB

$ docker volume ls | grep ecotracker
local     ecotracker_api-logs
local     ecotracker_mssql-data-staging

$ docker network ls | grep ecotracker
502c52ee1b90   ecotracker_ecotracker-net   bridge    local
```

Figura 4 — Imagem ecotracker-api:local (243 MB), volumes nomeados e rede ecotracker-net.

6.3 Aplicação em staging

```
# Smoke test - staging (health + login JWT)
$ curl -s http://localhost:8080/health
Healthy

$ curl -s -X POST http://localhost:8080/api/Auth/login -d '{"username":"admin","password":"Admin@123"}'
{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzb2FwLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9uYW1lIjoiYWRtaW4iLCJodHRwOi8
    y5taWNyb3NvZnQuY29tL3dzLzIwMDgvMDYvaWRlbnRpdHkvY2xhaW1zL3JvbGUiOiJBZG1pbiIsImp0aSI6ImE3NjgyMDIzLWRhYjUtNDBhMC05ZTAxLTA0YzA1YmQxZWNlYyIsImlhdCI6MTc
    iwiZXhwIjoxNzkxOTQzOTYyLCJpc3MiOiJFY29UcmFja2VyIiwiYXVkIjoiRWNvVHJhY2tlclVzZXJzIn0.e
```

Figura 5 — /health respondendo Healthy e emissão de token JWT em staging.

```
# Consulta autenticada - staging
$ curl -s 'http://localhost:8080/api/CarbonEmissions?pageNumber=1&pageSize=3' -H 'Authorization: Bearer <JWT>'
{
    "items": [
        {
            "id": 3,
            "companyName": "AgroSustentavel SA",
            "sector": "Agroneg\u00f3cio",
            "emissionTonsCO2": 3200.0,
            "offsetTonsCO2": 1500.0,
            "netEmission": 1700.0,
            "referenceYear": 2024,
            "referenceMonth": 3,
            "description": "Emiss\u00f5es agr\u00edcolas Q1",
            "createdAt": "2024-03-05T00:00:00"
        },
        {
            "id": 2,
            "companyName": "GreenTech Ltda",
            "sector": "Tecnologia",
            "emissionTonsCO2": 250.0,
            "offsetTonsCO2": 100.0,
            "netEmission": 150.0,
            "referenceYear": 2024,
            "referenceMonth": 2,
            "description": "Emiss\u00f5es do datacenter",
            "createdAt": "2024-02-10T00:00:00"
        },
        {
            "id": 1,
            "companyName": "EcoCorp Brasil",
            "sector": "Energia",
            "emissionTonsCO2": 1500.5,
            "offsetTonsCO2": 300.0,
            "netEmission": 1200.5,
            "referenceYear": 2024,
            "referenceMonth": 1,
            "description": "Emiss\u00f5es do primeiro trimestre",
            "createdAt": "2024-01-15T00:00:00"
        }
    ],
    "page": 1,
```

Figura 6 — Consulta autenticada a /api/CarbonEmissions retornando dados do SQL Server.

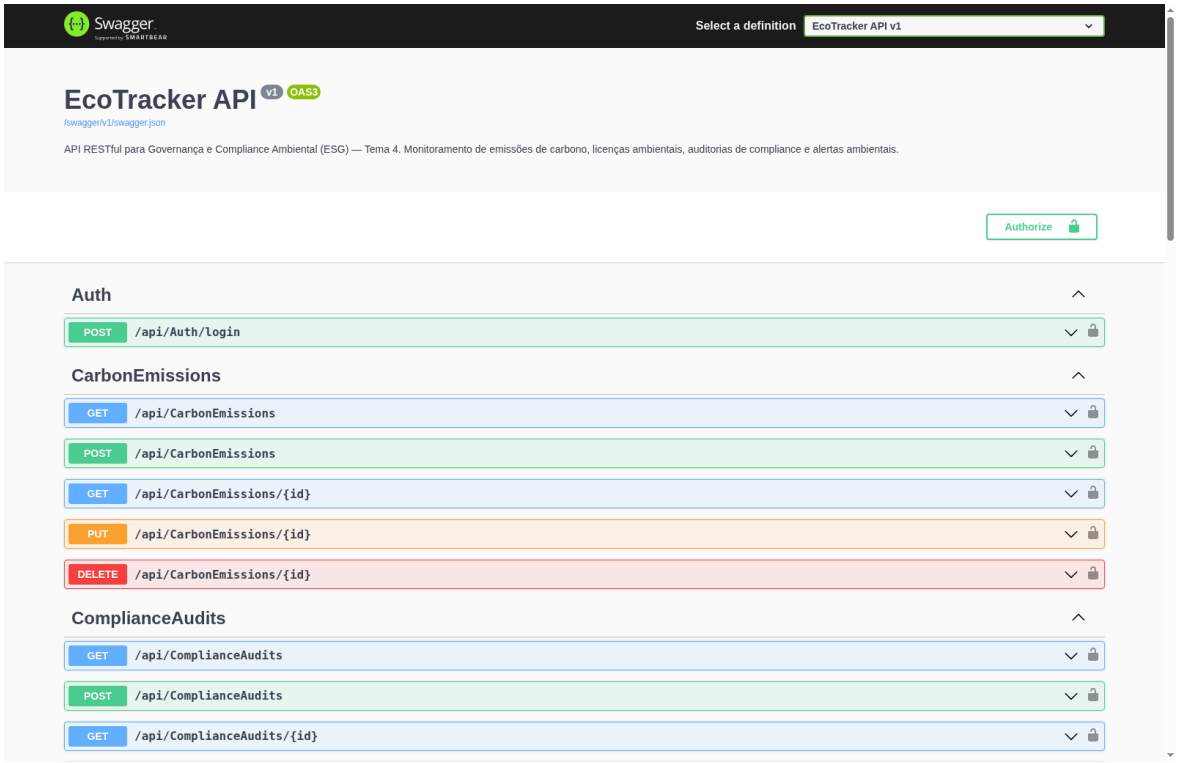


Figura 7 — Swagger da API em staging (http://localhost:8080/swagger).

6.4 Aplicação em produção

```
# Producao (porta 9090, Swagger desabilitado)
$ curl -s http://localhost:9090/health   # produção
Healthy

$ curl -s -o /dev/null -w '%{http_code}\n' http://localhost:9090/swagger/index.html   # Swagger desabilitado em produção
404

$ curl -s 'http://localhost:9090/api/EnvironmentalLicenses?pageNumber=1&pageSize=2' -H 'Authorization: Bearer <JWT>'
{"items":[{"id":1,"companyName":"EcoCorp Brasil","licenseNumber":"LIC-2024-001","licenseType":"Licença de Operação","issuingAuthority":"IBAMA","issueD
    -01-01T00:00:00","expirationDate":"2025-01-01T00:00:00","status":"Active","daysUntilExpiration":-651,"notes":null,"createdAt":"2024-01-01T00:00:00
    ,"companyName":"GreenTech Ltda","licenseNumber":"LIC-2024-002","licenseType":"Licença Prévia","issu

$ docker compose -f docker-compose.yml -f docker-compose.prod.yml ps
NAME                  IMAGE                                COMMAND                  SERVICE   CREATED              STATUS                    P
ecotracker-api-prod   ecotracker-api:local                 "dotnet EcoTracker.d…"   api       About a minute ago   Up 31 seconds (healthy)   0
    0->8080/tcp, [::]:9090->8080/tcp
ecotracker-db-prod    mcr.microsoft.com/mssql/server:2022-latest   "/opt/mssql/bin/laun…"   db        47 seconds ago       Up 37 seconds (healthy)   1
```

Figura 8 — Produção na porta 9090: /health saudável, Swagger retornando 404 (desabilitado) e banco sem porta publicada.

6.5 Pipeline no GitHub Actions

```
# Pipeline CI/CD no GitHub Actions
$ gh run view 38489723018   # pipeline completo no GitHub Actions


✓ devin/1791936616-devops CI/CD EcoTracker mde-figu/aspcore8#7 · 38489723018
Triggered via push about 5 minutes ago

JOBS
✓ Build in 58s (ID 115611791810)
✓ Testes automatizados in 34s (ID 115612095161)
✓ Build e push da imagem in 59s (ID 115612281150)
✓ Deploy em staging in 47s (ID 115612597403)
```

Figura 9 — Execução do pipeline: build, testes, imagem, deploy em staging e deploy em produção, todos aprovados.

```
# Job "Deploy em staging" no GitHub Actions
$ gh run view --job 115612597403 --log   # Deploy em staging (GitHub Actions)

 Container ecotracker-db-staging  Creating
 Container ecotracker-db-staging  Created
 Container ecotracker-api-staging  Creating
 Container ecotracker-api-staging  Created
 Container ecotracker-db-staging  Starting
 Container ecotracker-db-staging  Started
 Container ecotracker-db-staging  Waiting
 Container ecotracker-db-staging  Healthy
 Container ecotracker-api-staging  Starting
 Container ecotracker-api-staging  Started
API saudável em staging (tentativa 2)
{"items":[{"id":3,"companyName":"AgroSustentavel SA","sector":"Agronegócio","emissionTonsCO2":3200.0000,"offsetTonsCO2":1500.0000,"netEmission":170
Smoke test de staging concluído
NAME                    IMAGE                                                                                  COMMAND                  SERVICE   CREA
ecotracker-api-staging   ghcr.io/mde-figu/aspcore8/ecotracker-api:8b348e2d8fb3c0df993b2f172fd253edccc384ac   "dotnet EcoTracker.d…"   api       16
ecotracker-db-staging    mcr.microsoft.com/mssql/server:2022-latest                                             "/opt/mssql/bin/laun…"   db        16
 Container ecotracker-api-staging  Stopping
 Container ecotracker-api-staging  Stopped
```

Figura 10 — Job "Deploy em staging": stack no ar com a imagem do ghcr e smoke test concluído.

```
# Job "Deploy em producao" no GitHub Actions
$ gh run view --job 115612847299 --log   # Deploy em prod (GitHub Actions)

 Container ecotracker-db-prod  Creating
 Container ecotracker-db-prod  Created
 Container ecotracker-api-prod  Creating
 Container ecotracker-api-prod  Created
 Container ecotracker-db-prod  Starting
 Container ecotracker-db-prod  Started
 Container ecotracker-db-prod  Waiting
 Container ecotracker-db-prod  Healthy
 Container ecotracker-api-prod  Starting
 Container ecotracker-api-prod  Started
API saudável em prod (tentativa 2)
{"items":[{"id":3,"companyName":"AgroSustentavel SA","sector":"Agronegócio","emissionTonsCO2":3200.0000,"offsetTonsCO2":1500.0000,"netEmission":170
Smoke test de prod concluído
NAME                 IMAGE                                                                                COMMAND                  SERVICE   CREATED
ecotracker-api-prod   ghcr.io/mde-figu/aspcore8/ecotracker-api:8b348e2d8fb3c0df993b2f172fd253edccc384ac   "dotnet EcoTracker.d…"   api       17 se
ecotracker-db-prod    mcr.microsoft.com/mssql/server:2022-latest                                           "/opt/mssql/bin/laun…"   db        17 se
 Container ecotracker-api-prod  Stopping
 Container ecotracker-api-prod  Stopped
```

Figura 11 — Job "Deploy em produção": mesma imagem promovida, health check e smoke test aprovados.

```
# Validacao do compose e logs da API
$ docker compose -f docker-compose.yml -f docker-compose.staging.yml config --quiet && echo OK
staging OK
prod OK

$ docker compose logs api --tail 12
ecotracker-api-prod  | warn: Microsoft.AspNetCore.DataProtection.Repositories.FileSystemXmlRepository[60]
ecotracker-api-prod  |       Storing keys in a directory '/home/ecotracker/.aspnet/DataProtection-Keys' that may not be persisted outside of the conta
    ected data will be unavailable when container is destroyed. For more information go to https://aka.ms/aspnet/dataprotectionwarning
ecotracker-api-prod  | warn: Microsoft.AspNetCore.DataProtection.KeyManagement.XmlKeyManager[35]
ecotracker-api-prod  |       No XML encryptor configured. Key {8e1c90e2-2e41-4d26-a33a-c7d6fd32b0ab} may be persisted to storage in unencrypted form.
ecotracker-api-prod  | warn: Microsoft.AspNetCore.Hosting.Diagnostics[15]
ecotracker-api-prod  |       Overriding HTTP_PORTS '8080' and HTTPS_PORTS ''. Binding to values defined by URLS instead 'http://+:8080'.
ecotracker-api-prod  | warn: Microsoft.AspNetCore.DataProtection.Repositories.FileSystemXmlRepository[60]
ecotracker-api-prod  |       Storing keys in a directory '/home/ecotracker/.aspnet/DataProtection-Keys' that may not be persisted outside of the conta
    ected data will be unavailable when container is destroyed. For more information go to https://aka.ms/aspnet/dataprotectionwarning
ecotracker-api-prod  | warn: Microsoft.AspNetCore.Hosting.Diagnostics[15]
ecotracker-api-prod  |       Overriding HTTP_PORTS '8080' and HTTPS_PORTS ''. Binding to values defined by URLS instead 'http://+:8080'.
```

Figura 12 — Validação dos arquivos de Compose (staging e produção) e logs da API.

7. Desafios encontrados e soluções

Desafio	Solução adotada
A API subia antes do SQL Server aceitar conexões e quebrava no EnsureCreated()	healthcheck no serviço db + depends_on: service_healthy e laço de retry no startup (Database:StartupRetries)
O container serve apenas HTTP, mas a aplicação forçava redirecionamento para HTTPS	Redirecionamento tornou-se configurável (DisableHttpsRedirection=true na imagem); TLS fica a cargo do proxy
Swagger só existia em Development, impedindo validar a API em staging	Nova chave EnableSwagger: true em staging, false em produção
Não havia como o Docker e o pipeline saberem se a aplicação subiu de fato	Endpoint anônimo /health (Health Checks) usado pelo HEALTHCHECK da imagem e pelos jobs de deploy
O override de produção não removia a porta publicada do banco (listas do Compose são mescladas)	Uso da tag !reset [] em ports no docker-compose.prod.yml
Duplicação da lógica de deploy entre os dois ambientes	Composite action .github/actions/deploy-compose parametrizada por ambiente, override, porta e segredos
Risco de versionar senhas	.env no .gitignore, apenas .env.example versionado e segredos lidos de secrets do repositório

8. Como executar

```
cp .env.example .env            # ajuste senhas e portas
docker compose up -d --build    # sobe api + db
curl http://localhost:8080/health
# Swagger: http://localhost:8080/swagger   (login: admin / Admin@123)
docker compose down -v          # derruba a stack e remove os volumes
```

9. Checklist da entrega

- [ok] Projeto compactado em .zip organizado
- [ok] Dockerfile funcional (multi-stage, usuário não-root, healthcheck)
- [ok] docker-compose.yml com volumes, variáveis de ambiente e rede (+ overrides de staging e produção)
- [ok] Pipeline com build, testes e deploy automatizados (GitHub Actions)
- [ok] README com instruções de execução e prints
- [ok] Documentação técnica com evidências (este documento)
- [ok] Deploy realizado em staging e em produção